

# Optimizing High-Throughput GPU-to-SSD Data Writes

Writing Fast from GPU to SSD

Rosemarie Streefkerk

Supervisor: Charles Prouveur

Laboratory: Maison de la Simulation, CEA

August 25, 2026



université  
PARIS-SACLAY

# Introduction

Compute is outpacing storage

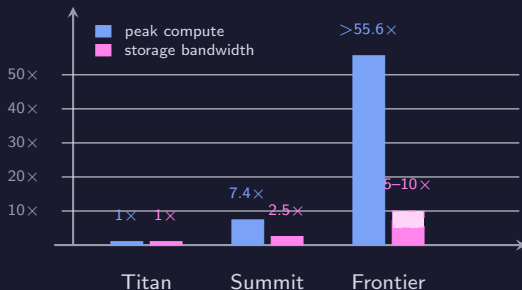
**Computation is accelerating faster than storage.**

## Seismic-wave simulation (2026)

Saves snapshots every 10 timesteps (~150 MB each):

- ▶ up to **90.6%** of total runtime spent on I/O

growth vs. Titan (×)



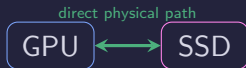
Source: Oak Ridge National Laboratory Titan/Summit/Frontier specs

# Direct to SSD solutions

The fastest way to write from GPU to SSD is to have a direct link

---

## GPUDirect Storage

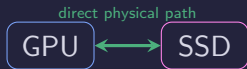


Requires: compatible GPU, storage topology, drivers, and filesystem support.

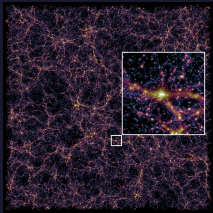
# Direct to SSD solutions

The fastest way to write from GPU to SSD is to have a direct link

## GPUDirect Storage



Requires: compatible GPU, storage topology, drivers, and filesystem support.



## Frontier-E, November 2025

A **four-trillion-particle** cosmology simulation used Frontier's two node-local NVMe SSDs per compute node for its fast I/O tier.

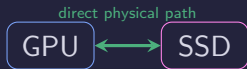
>100 PB written  
~5.1 hours of I/O  
~3% of total runtime

Frontiere et al., SC '25 (paper and DOI)

# Direct to SSD solutions

The fastest way to write from GPU to SSD is to have a direct link

## GPUDirect Storage



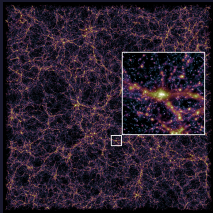
Requires: compatible GPU, storage topology, drivers, and filesystem support.

## Data Processing Unit



Moves data handling & I/O away from CPU.

Requires: dedicated accelerator hardware and suitable software stack.



## Frontier-E, November 2025

A **four-trillion-particle** cosmology simulation used Frontier's two node-local NVMe SSDs per compute node for its fast I/O tier.

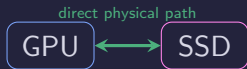
>100 PB written  
~5.1 hours of I/O  
~3% of total runtime

Frontiere et al., SC '25 (paper and DOI)

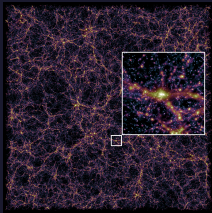
# Direct to SSD solutions

The fastest way to write from GPU to SSD is to have a direct link

## GPUDirect Storage



Requires: compatible GPU, storage topology, drivers, and filesystem support.



## Frontier-E, November 2025

A four-trillion-particle cosmology simulation used Frontier's two node-local NVMe SSDs per compute node for its fast I/O tier.

>100 PB written  
~5.1 hours of I/O  
~3% of total runtime

Frontiere et al., SC '25 (paper and DOI)

## Data Processing Unit



Moves data handling & I/O away from CPU.

Requires: dedicated accelerator hardware and suitable software stack.

## rocm-xio



Lets GPU kernel code issue IO directly to NVMe, RDMA, or SDMA endpoints.

Requires: ROCm-capable GPU; early-access preview, not yet production-ready.

# Roadmap

How can we maximize I/O throughput and limit the SSD-write bottleneck?

---

1. **Current available technologies**

GPU-to-CPU copies and SSD writes

# Roadmap

How can we maximize I/O throughput and limit the SSD-write bottleneck?

---

1. **Current available technologies**

GPU-to-CPU copies and SSD writes

2. **Scalable solutions**

Benchmark results of HDF5 and MPI I/O solutions

# Roadmap

How can we maximize I/O throughput and limit the SSD-write bottleneck?

---

1. **Current available technologies**

GPU-to-CPU copies and SSD writes

2. **Scalable solutions**

Benchmark results of HDF5 and MPI I/O solutions

3. **Conclusion: I/O trade-offs**

Feasibility, implementation and trade-offs

## PART 1

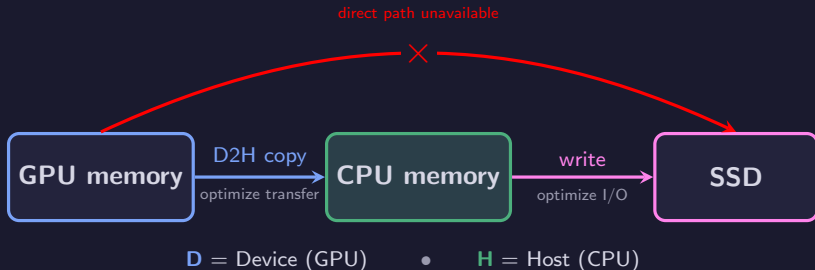
# Current available technologies

GPU-to-CPU copies and SSD writes

---

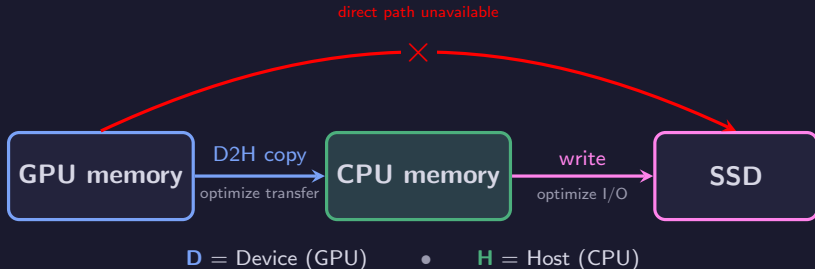
# Optimizing GPU to CPU copies

We can't bypass the CPU but we can optimize CPU writes



# Optimizing GPU to CPU copies

We can't bypass the CPU but we can optimize CPU writes

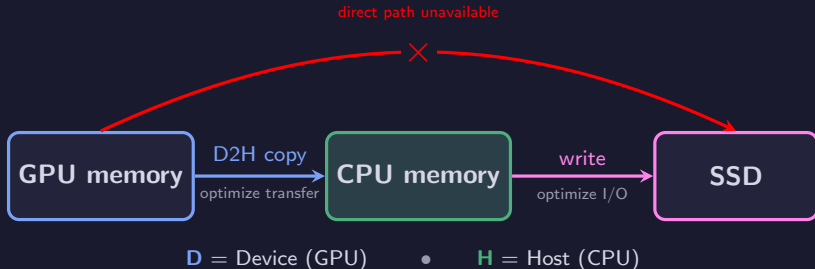


## Allocate wisely

The way we allocate host-side memory can impact performance.

# Optimizing GPU to CPU copies

We can't bypass the CPU but we can optimize CPU writes



## Allocate wisely

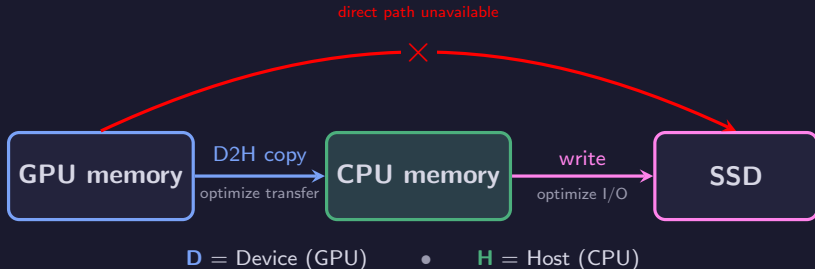
The way we allocate host-side memory can impact performance.

## Overlap stages

Hide transfer and write latency behind computation.

# Optimizing GPU to CPU copies

We can't bypass the CPU but we can optimize CPU writes



## Allocate wisely

The way we allocate host-side memory can impact performance.

## Overlap stages

Hide transfer and write latency behind computation.

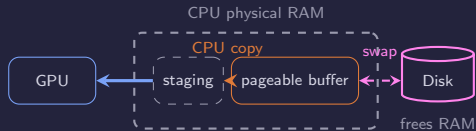
## Write efficiently

Select a suitable parallel I/O strategy to optimize SSD writes.

# Allocating host-side memory

The type of allocation has an impact on performance

## Pageable: an extra copy



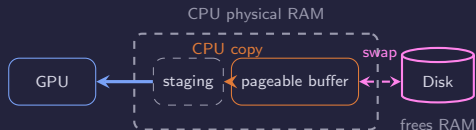
```
// Regular host allocation
float *buffer = malloc(bytes);

free(buffer);
```

# Allocating host-side memory

The type of allocation has an impact on performance

## Pageable: an extra copy



```
// Regular host allocation
float *buffer = malloc(bytes);

free(buffer);
```

## Pinned: direct transfer



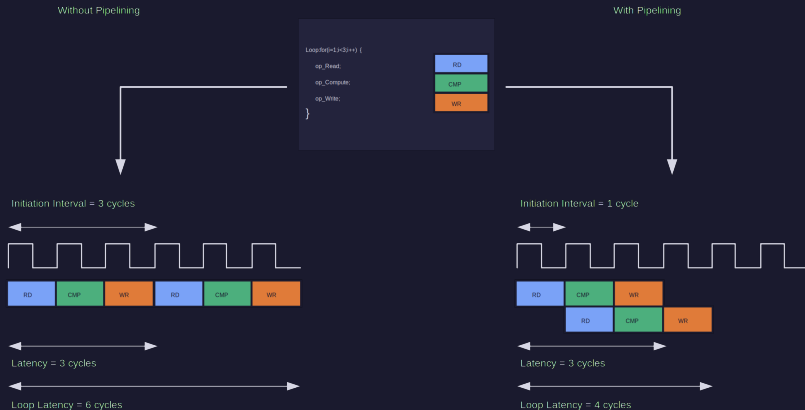
```
// Page-locked host allocation
float *buffer;
cudaMallocHost(&buffer, bytes);

cudaFreeHost(buffer);
```

→ **Allocate pinned memory sparingly:** It cannot be swapped so it reduces RAM available to the OS.

# Keeping the I/O pipeline busy

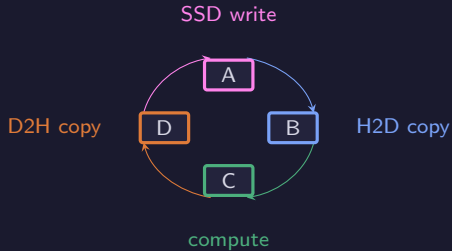
Split the data into chunks so that independent resources can work concurrently



# Keeping the I/O pipeline busy

Using batching and a ring buffer

## Ring buffering

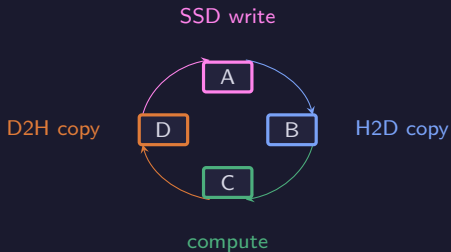


# Keeping the I/O pipeline busy

Using batching and a ring buffer

---

## Ring buffering

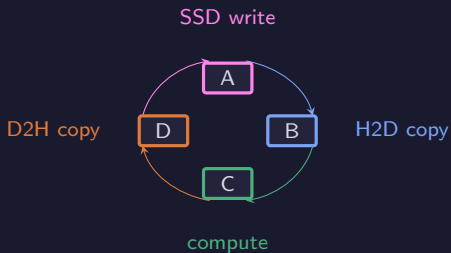


- ▶ Overlaps compute, copies, and I/O
- ▶ Absorbs short performance spikes
- ▶ Needs careful synchronization

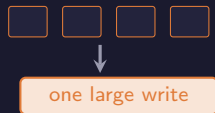
# Keeping the I/O pipeline busy

## Using batching and a ring buffer

### Ring buffering



### Batching

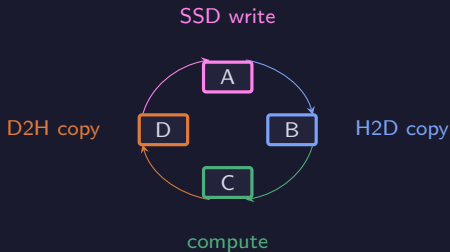


- ▶ Overlaps compute, copies, and I/O
- ▶ Absorbs short performance spikes
- ▶ Needs careful synchronization

# Keeping the I/O pipeline busy

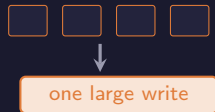
## Using batching and a ring buffer

### Ring buffering



- ▶ Overlaps compute, copies, and I/O
- ▶ Absorbs short performance spikes
- ▶ Needs careful synchronization

### Batching



- ▶ Amortizes per-write call setup, metadata/locking, and request-scheduling overhead
- ▶ Fewer I/O calls and better effective bandwidth
- ▶ Adds latency and memory pressure

## PART 2

# Scalable solutions

Benchmark results of HDF5 and MPI I/O solutions

---

# Benchmark methodology

Experimental setup and how to read the results

---

## Architecture

- ▶ **AMD Adastra MI250X nodes**
- ▶ 8 GPUs/node • 1 MPI rank/GPU
- ▶ 1–32 ranks tested

# Benchmark methodology

Experimental setup and how to read the results

---

## Architecture

- ▶ **AMD Adastra MI250X nodes**
- ▶ 8 GPUs/node • 1 MPI rank/GPU
- ▶ 1–32 ranks tested

## Timed regions

- ▶ **D2H**: HIP events around `hipMemcpyAsync`
- ▶ **SSD**: `MPI_Wtime` around write + flush/`fsync`
- ▶ Untimed warm-up run before each timed region
- ▶ Large, page-aligned chunks for direct I/O
- ▶ HDF5 objects and metadata created before timing

Excluded: allocation • initialization • object creation • verification

# Benchmark methodology

## Experimental setup and how to read the results

---

### Architecture

- ▶ **AMD Adastr** MI250X nodes
- ▶ 8 GPUs/node • 1 MPI rank/GPU
- ▶ 1–32 ranks tested

### Timed regions

- ▶ **D2H**: HIP events around `hipMemcpyAsync`
- ▶ **SSD**: `MPI_Wtime` around write + flush/`fsync`
- ▶ Untimed warm-up run before each timed region
- ▶ Large, page-aligned chunks for direct I/O
- ▶ HDF5 objects and metadata created before timing

Excluded: allocation • initialization • object creation • verification

### Bandwidth calculation

Single rank:  $B = \text{bytes}/t$

Parallel writes:  $B_{\text{agg}} = N_{\text{ranks}} \text{ bytes/rank} / \max_r(t_r)$

- ▶ Aggregate bandwidth; slowest rank sets completion
- ▶ 10 runs: mean • standard deviation • variance
- ▶ Median • minimum • maximum
- ▶ Verification status: written data checked

# Benchmark methodology

## Experimental setup and how to read the results

### Architecture

- ▶ **AMD Adastr** MI250X nodes
- ▶ 8 GPUs/node • 1 MPI rank/GPU
- ▶ 1–32 ranks tested

### Timed regions

- ▶ **D2H**: HIP events around `hipMemcpyAsync`
- ▶ **SSD**: `MPI_Wtime` around write + flush/`fsync`
- ▶ Untimed warm-up run before each timed region
- ▶ Large, page-aligned chunks for direct I/O
- ▶ HDF5 objects and metadata created before timing

Excluded: allocation • initialization • object creation • verification

### Bandwidth calculation

Single rank:  $B = \text{bytes}/t$

Parallel writes:  $B_{\text{agg}} = N_{\text{ranks}} \text{ bytes/rank} / \max_r(t_r)$

- ▶ Aggregate bandwidth; slowest rank sets completion
- ▶ 10 runs: mean • standard deviation • variance
- ▶ Median • minimum • maximum
- ▶ Verification status: written data checked

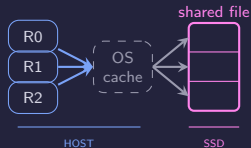
### Scope and caveats

- ▶ Synthetic data • contiguous • page-aligned
- ▶ More ranks  $\Rightarrow$  more total data
- ▶ Buffered/direct: different caching

# Individual and collective writes

## Rank coordination in MPI-IO

### Individual writes



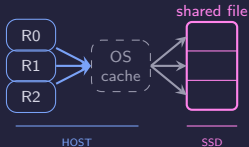
- ▶ Each rank issues its own request
- ▶ Low coordination; possible contention

```
// MPI-IO
MPI_File_write_at(file, offset, buffer, count, MPI_BYTE, &status);
// Parallel HDF5
H5Pset_dxpl_mpio(dxpl, H5FD_MPIO_INDEPENDENT);
H5Dwrite(dset, type, mspace, fspace, dxpl, buffer);
```

# Individual and collective writes

## Rank coordination in MPI-IO

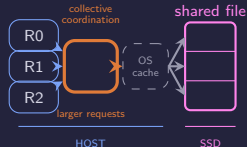
### Individual writes



- ▶ Each rank issues its own request
- ▶ Low coordination; possible contention

```
// MPI-IO
MPI_File_write_at(file, offset, buffer, count, MPI_BYTE, &status);
// Parallel HDF5
H5Pset_dxpl_mpio(dxpl, H5FD_MPIO_INDEPENDENT);
H5Dwrite(dset, type, mspace, fspace, dxpl, buffer);
```

### Collective writes

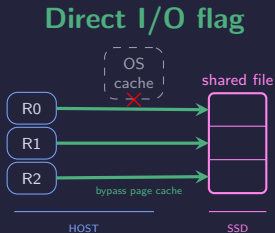


- ▶ Ranks enter the operation together
- ▶ MPI may aggregate/reorder requests
- ▶ Fewer, larger writes; synchronization cost

```
// MPI-IO
MPI_File_write_at_all(file, offset, buffer, count, MPI_BYTE, &status);
// Parallel HDF5
H5Pset_dxpl_mpio(dxpl, H5FD_MPIO_COLLECTIVE);
H5Dwrite(dset, type, mspace, fspace, dxpl, buffer);
```

# Direct I/O

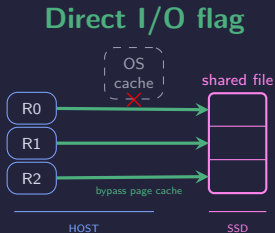
## Bypassing the OS page cache



- ▶ Bypasses the OS page cache
- ▶ Requires aligned buffer, offset and size
- ▶ Less cache pollution; stricter constraints

# Direct I/O

Bypassing the OS page cache



- ▶ Bypasses the OS page cache
- ▶ Requires aligned buffer, offset and size
- ▶ Less cache pollution; stricter constraints

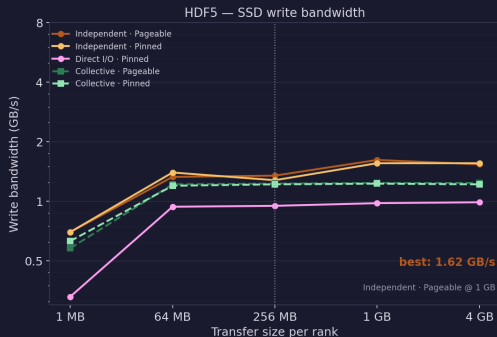
## MPI-IO and HDF5 implementation

```
MPI_Info info;  
MPI_Info_create(&info);  
MPI_Info_set(info, "direct_io", "true");  
  
// MPI-IO  
MPI_File_open(comm, path, mode,  
               info, &mpi_file);  
  
// Parallel HDF5: pass the same MPI_Info  
hid_t fapl = H5Pcreate(H5P_FILE_ACCESS);  
H5Pset_fapl_mpio(fapl, comm, info);  
hid_t file = H5Fopen(path, H5F_ACC_RDWR, fapl);
```

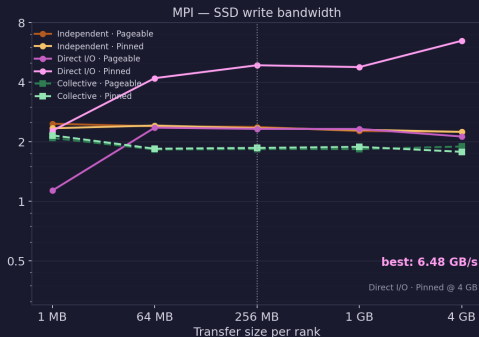
# SSD write bandwidth: CPU → SSD

Adastra MI250X • 1 node • 1 GPU/GCD (1 rank) • Lustre default striping (2 × 1MiB)

## HDF5



## MPI



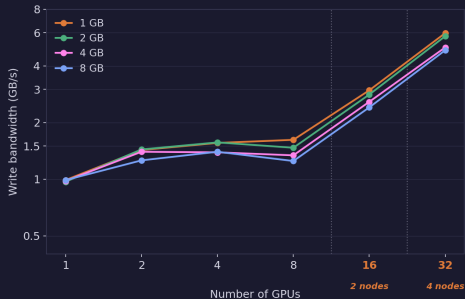
Performance gap between pageable and pinned direct I/O: pinned memory is guaranteed to be page-aligned which is required by the direct I/O path. When pageable memory is not aligned, direct I/O silently falls back to buffered writes.

# SSD writes: scaling comparison

Adastra MI250X • Lustre default striping (2 × 1MiB)

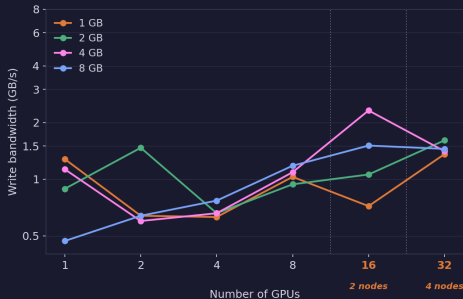
## HDF5 SSD

write bandwidth vs #GPUs — Pinned · Independent



## MPI SSD

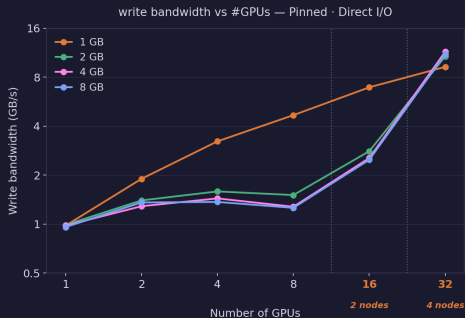
write bandwidth vs #GPUs — Pinned · Collective



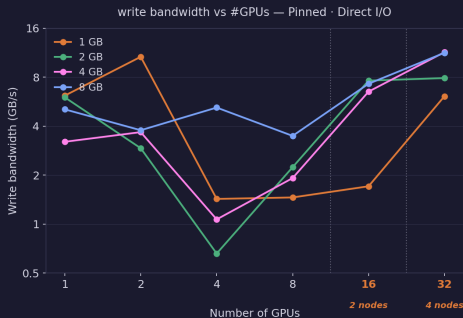
# SSD writes: adding the direct\_io flag

Adastra MI250X • Lustre default striping (2 x 1MiB)

## HDF5 SSD



## MPI SSD



# Lustre striping

Taking advantage of the distributed file system architecture

---

## One file, split into stripes



each numbered block = one stripe, sized `stripe_size` bytes  
colors repeat because stripes assign round-robin once all OSTs are used

# Lustre striping

Taking advantage of the distributed file system architecture

---

## One file, split into stripes



each numbered block = one stripe, sized `stripe_size` bytes  
colors repeat because stripes assign round-robin once all OSTs are used

`stripe_count` = 4 **OSTs share this file**

OST = Object Storage Target: a Lustre storage target that holds file data



# Lustre striping

Taking advantage of the distributed file system architecture

---

## One file, split into stripes



each numbered block = one stripe, sized `stripe_size` bytes  
colors repeat because stripes assign round-robin once all OSTs are used

`stripe_count` = 4 **OSTs share this file**

**OST** = Object Storage Target: a Lustre storage target that holds file data



More OSTs  $\Rightarrow$  more potential parallel bandwidth,  
but **greater exposure to shared-system contention**.

# Setting Lustre striping

Configure the output layout from the job script or the application

---

## In the Slurm script

```
#!/bin/bash
#SBATCH --job-name=gpu-io

STRIPE_COUNT=8
STRIPE_SIZE=4M
OUTDIR="$SCRATCH/run_${SLURM_JOB_ID}"

mkdir -p "$OUTDIR"
lfs setstripe -c "$STRIPE_COUNT" \
             -S "$STRIPE_SIZE" \
             "$OUTDIR"

# New files inherit the directory layout
srun ./benchmark "$OUTDIR/output.dat"
```

Useful when all output files share one layout.

# Setting Lustre striping

Configure the output layout from the job script or the application

## In the Slurm script

```
#!/bin/bash
#SBATCH --job-name=gpu-io

STRIPE_COUNT=8
STRIPE_SIZE=4M
OUTDIR="$SCRATCH/run_${SLURM_JOB_ID}"

mkdir -p "$OUTDIR"
lfs setstripe -c "$STRIPE_COUNT" \
              -S "$STRIPE_SIZE" \
              "$OUTDIR"

# New files inherit the directory layout
srun ./benchmark "$OUTDIR/output.dat"
```

Useful when all output files share one layout.

## Directly in the C code

```
#include <fcntl.h>
#include <unistd.h>
#include <lustre/lustreapi.h>

const unsigned long long stripe_size = 4ULL << 20;
const int stripe_count = 8;

int fd = llapi_file_open(
    path, O_CREAT | O_WRONLY, 0644,
    stripe_size, -1, stripe_count, 0);

if (fd < 0)
    /* handle -errno */;
else
    close(fd);

// The file can now be opened by MPI-IO or HDF5.
```

Link with `-llustreapi`; `-1` selects the starting OST automatically.

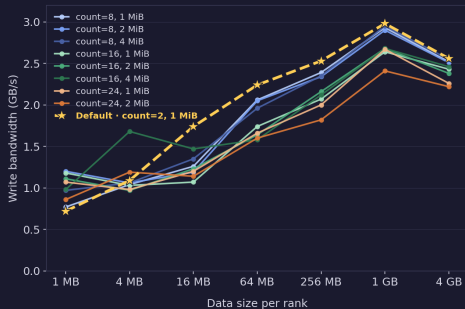
Set the layout before writing: striping cannot be changed once file data has been allocated.

# Stripe configuration impact on HDF5 I/O

Adastra MI250X • 16 GPUs (ranks)

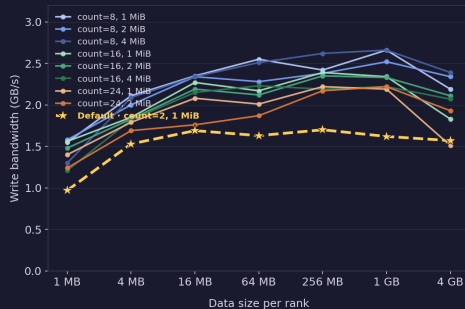
## Pinned Independent

write bandwidth vs data size/rank — 2 nodes / 16 GPUs, all tested stripe configs



## Pinned Collective

write bandwidth vs data size/rank — 2 nodes / 16 GPUs, all tested stripe configs

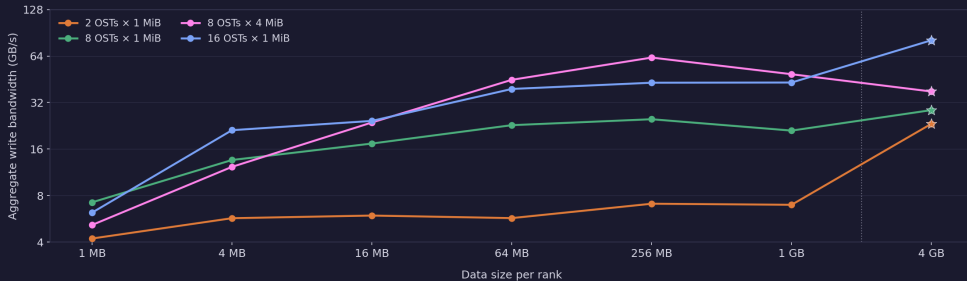


# Stripe configuration impact on MPI I/O performance

Adastra MI250X • 16 GPUs (ranks)

## MPI Direct I/O — Lustre striping sweep

Pinned memory · 16 ranks · aggregate SSD write bandwidth



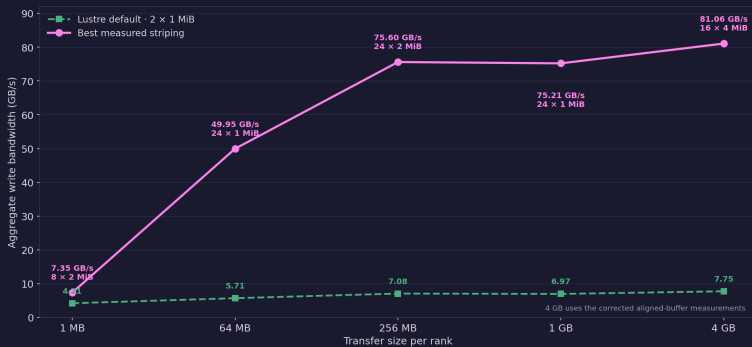
In orange = Lustre default configuration, used for all previous results.

# Adapting stripe configuration to size of data

Adastra MI250X • 16 GPUs (ranks)

## MPI Direct I/O

Lustre default vs best measured striping · 16 ranks · pinned memory



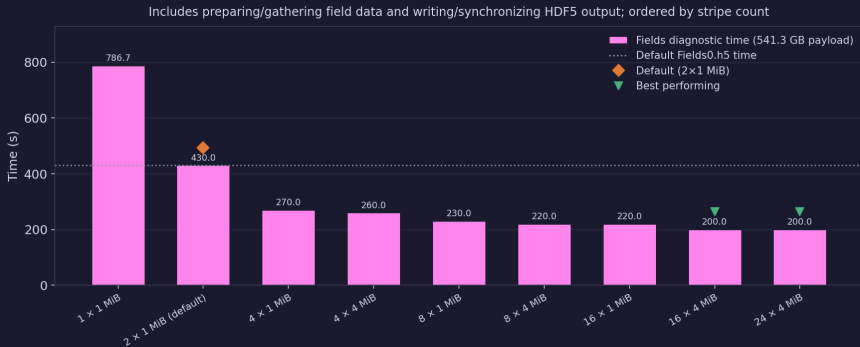
Adastra scratch peak write bandwidth: **844 GB/s** cluster-wide across **40 OSTs**

# Application-level striping: Smilei

Adastra MI250X • 4 nodes, 32 GPUs (MPI ranks)

2D plasma simulation, field diagnostics written periodically to a single 541.3 GB HDF5 file.

## Fields diagnostic time



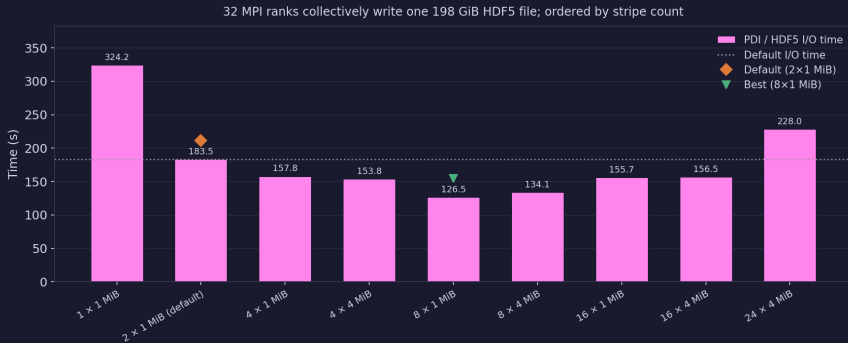
Performance improves strongly versus 1×1 MiB, then plateaus around the best 16×4 MiB / 24×4 MiB configurations

# Application-level striping: PDI / HDF5

Adastra Genoa • 32 MPI ranks on 32 CPU cores • single shared HDF5 file

MPI heat-equation-style benchmark from the PDI NumPEx tutorial, very I/O-dominated ( 96–98% of runtime).

## PDI / HDF5 I/O time



Performance improves up to 8 stripes, then degrades again as stripe count keeps increasing

## PART 3

# Conclusion: I/O trade-offs

Feasibility, implementation and trade-offs

---

# Conclusion: choosing an optimal I/O strategy

The best combination of technologies depends on workload and system

---

## When to choose MPI-IO or HDF5

### MPI-IO

- ✓ Large parallel transfers
- ✓ Fragmented accesses

### HDF5

# Conclusion: choosing an optimal I/O strategy

The best combination of technologies depends on workload and system

---

## When to choose MPI-IO or HDF5

### MPI-IO

- ✓ Large parallel transfers
- ✓ Fragmented accesses
- ✗ Self-describing format required
- ✗ Rich metadata

### HDF5

# Conclusion: choosing an optimal I/O strategy

The best combination of technologies depends on workload and system

---

## When to choose MPI-IO or HDF5

### MPI-IO

- ✓ Large parallel transfers
- ✓ Fragmented accesses
- ✗ Self-describing format required
- ✗ Rich metadata

### HDF5

- ✓ Portable, self-describing datasets
- ✓ Analysis, metadata and selective access

# Conclusion: choosing an optimal I/O strategy

The best combination of technologies depends on workload and system

---

## When to choose MPI-IO or HDF5

### MPI-IO

- ✓ Large parallel transfers
- ✓ Fragmented accesses
- ✗ Self-describing format required
- ✗ Rich metadata

### HDF5

- ✓ Portable, self-describing datasets
- ✓ Analysis, metadata and selective access
- ✗ Lowest overhead is the priority
- ✗ Full control of file layout required

# Conclusion: choosing an optimal I/O strategy

The best combination of technologies depends on workload and system

---

## When to choose MPI-IO or HDF5

### MPI-IO

- ✓ Large parallel transfers
- ✓ Fragmented accesses
- ✗ Self-describing format required
- ✗ Rich metadata

### HDF5

- ✓ Portable, self-describing datasets
- ✓ Analysis, metadata and selective access
- ✗ Lowest overhead is the priority
- ✗ Full control of file layout required

## When to choose direct I/O

- ✓ Large, contiguous and aligned transfers
- ✓ Temporary bulk output with no immediate reread

# Conclusion: choosing an optimal I/O strategy

The best combination of technologies depends on workload and system

---

## When to choose MPI-IO or HDF5

### MPI-IO

- ✓ Large parallel transfers
- ✓ Fragmented accesses
- ✗ Self-describing format required
- ✗ Rich metadata

### HDF5

- ✓ Portable, self-describing datasets
- ✓ Analysis, metadata and selective access
- ✗ Lowest overhead is the priority
- ✗ Full control of file layout required

## When to choose direct I/O

- ✓ Large, contiguous and aligned transfers
- ✓ Temporary bulk output with no immediate reread
- ✗ Small, irregular or unaligned accesses
- ✗ Data likely to be reread soon through the page cache

# Feasibility and implementation

Practical effort and integration choices

---

- ▶ **Pinned memory: easy** → one-line allocation change.

# Feasibility and implementation

Practical effort and integration choices

---

- ▶ **Pinned memory: easy** → one-line allocation change.
- ▶ **Ring buffer: higher effort** → enables major gains from overlap, but incorrect buffer reuse can silently corrupt data.

# Feasibility and implementation

Practical effort and integration choices

---

- ▶ **Pinned memory: easy** → one-line allocation change.
- ▶ **Ring buffer: higher effort** → enables major gains from overlap, but incorrect buffer reuse can silently corrupt data.
- ▶ **Lustre striping: easy** → one line in the Slurm script, or a small function selecting stripes from data size and MPI rank count.

# Feasibility and implementation

Practical effort and integration choices

---

- ▶ **Pinned memory: easy** → one-line allocation change.
- ▶ **Ring buffer: higher effort** → enables major gains from overlap, but incorrect buffer reuse can silently corrupt data.
- ▶ **Lustre striping: easy** → one line in the Slurm script, or a small function selecting stripes from data size and MPI rank count.
- ▶ **Keep the existing MPI/HDF5 stack:** tune striping, check whether direct I/O is active or usable, and pipeline the transfer stages.

# Thank you!

Any questions?